

1. ¿Cuál es la principal diferencia entre una arquitectura SOA y una arquitectura de microservicios?

SOA utiliza un bus central (ESB) para coordinar servicios y transformar datos, mientras que los microservicios eliminan esa dependencia y cada servicio funciona de forma independiente con su propia lógica y base de datos. Los microservicios usan normalmente REST/JSON o gRPC y favorecen la escalabilidad y autonomía de equipos.

2. Explique por qué el Enterprise Service Bus (ESB) puede convertirse en un riesgo crítico dentro de una arquitectura SOA.

El ESB puede convertirse en un punto único de fallo. Si el bus central deja de funcionar, toda la organización puede quedar paralizada. Además, concentra demasiada lógica de negocio, generando acoplamiento de infraestructura.

3. ¿Qué ventajas aporta la descentralización de datos en microservicios frente a una base de datos compartida?

Cada microservicio posee su propia base de datos, lo que reduce el acoplamiento entre módulos y permite modificar servicios sin afectar al resto. También mejora la escalabilidad y la independencia tecnológica.

4. Defina el concepto de resiliencia en APIs REST y relacione su importancia con las falsas suposiciones sobre redes distribuidas.

La resiliencia consiste en diseñar APIs capaces de soportar fallos de red, latencia o cambios de infraestructura sin colapsar. Surge porque muchas veces se asume erróneamente que la red es fiable, rápida y segura, cuando realmente no lo es.

5. Explique qué es el Model Context Protocol (MCP) y cómo facilita la interacción entre LLMs y herramientas externas.

El Model Context Protocol (MCP) es un estándar que permite que un modelo de lenguaje (LLM) interactúe con herramientas externas de forma estructurada. Con ello un LLM puede leer información, ejecutar acciones y escribir resultados usando un protocolo en común.

6. Diferencie entre “Tool Use” y un servidor MCP dentro de una arquitectura basada en IA.

El Tool use es la capacidad puntual de un LLM para utilizar herramientas concretas. Un servidor MCP, en cambio, es una infraestructura completa que organiza la comunicación entre el modelo y las herramientas.

7. ¿Por qué la Ingeniería Inversa es fundamental para la interoperabilidad y el análisis de malware?

Esto es debido que la ingeniería inversa permite analizar sistemas ya construidos para comprender su funcionamiento interno. Es clave para lograr interoperabilidad entre sistemas cerrados o que no proporcionan documentación API o interna y así poder recuperar Software Legacy.

8. Compare Ingeniería Directa e Ingeniería Inversa indicando sus diferencias en el flujo de abstracción.

La Ingeniería Directa parte de requisitos y diseños abstractos para crear software. La Ingeniería Inversa hace el proceso contrario: parte de un sistema ya implementado y lo descompone para descubrir su diseño y funcionamiento.

9. ¿Qué papel cumple el estándar ISO 19506 (KDM) en la gobernanza de sistemas legacy?

La ISO 19506 define un estándar para representar el conocimiento de sistemas software legacy. Facilita la gobernanza TI, el análisis de dependencias y la modernización de sistemas antiguo.

10. Explique cómo el derecho a la interoperabilidad limita legalmente ciertas restricciones impuestas por contratos EULA.

La ley permite realizar ingeniería inversa cuando es necesaria para garantizar interoperabilidad entre sistemas. Por ello, algunas restricciones impuestas por contratos EULA no pueden prevalecer sobre el derecho legal reconocido.

11. ¿Cuál es el objetivo principal de la norma ISO/IEC 27034 en la securización de aplicaciones?

La norma ISO/IEC 27034 busca integrar la seguridad durante el ciclo de vida del software, proporcionando directrices para desarrollar aplicaciones seguras desde el diseño hasta el mantenimiento.

12. Defina el concepto de ONF (Organizational Normative Framework) y explique su importancia en el ciclo de vida seguro del software.

El ONF (Organizational Normative Framework) es un marco organizativo que reúne políticas, reglas y controles de seguridad aplicables en el desarrollo software. Su función es garantizar que la seguridad se mantenga durante todo el ciclo de vida de la aplicación.

13. Relacione el S-SDLC con OWASP Top 10 y explique cómo contribuye a reducir vulnerabilidades críticas.

El Software Development Life Cycle incorpora controles de seguridad en cada fase del desarrollo software. Esto ayuda a prevenir vulnerabilidades descritas en OWASP Top 10, como inyecciones SQL o fallos de autenticación.

14. Compare las herramientas SAST y DAST indicando ventajas, desventajas y complementariedad dentro de DevSecOps.

SAST analiza el código fuente de forma estática antes de ejecutar la aplicación, detectando errores tempranos. DAST analiza la aplicación en ejecución simulando ataques reales. SAST encuentra fallos internos y DAST detecta vulnerabilidades en tiempo real; ambos son complementarios en DevSecOps.

15. Explique la filosofía “Docs as Code” y sus ventajas frente a la documentación tradicional basada en Word.

Docs as Code propone tratar la documentación igual que el código fuente, utilizando Git, Markdown, pull requests y CI/CD. Esto mejora la trazabilidad, automatización y colaboración frente a documentos Word tradicionales.

16. ¿Qué relación existe entre la mantenibilidad definida en ISO 25010 y el llamado “bus factor”?

La mantenibilidad en ISO 25010 mide la facilidad para modificar y comprender un sistema. El “bus factor” representa el riesgo de que el conocimiento dependa de pocas personas; buena documentación y diseño reducen ese riesgo

17. Describa cómo OpenAPI y Swagger funcionan como contrato técnico entre servicios y equipos de desarrollo

OpenAPI y Swagger definen formalmente cómo funciona una Api, especificando endpoints, parametros y respuestas. Actúan como contrato técnico entre equipos, evitando inconsistencias e incompatibilidades.

18. Explique cómo TDD mejora las subcaracterísticas de Testabilidad y Modificabilidad definidas en ISO/IEC 25010.

TDD obliga a escribir pruebas antes del código, lo que genera software más modular y desacoplado. Esto mejora la testabilidad, analizabilidad y modificabilidad definidas en ISO/IEC 25010.

19. ¿Por qué el uso de IA generativa en desarrollo software puede incrementar la deuda técnica si no existen mecanismos de validación?

La IA puede generar código funcional pero difícil de entender o mantener, aumentando la carga cognitiva y la deuda técnica. Por ello, son necesarios tests, code reviews humanos y analizadores estáticos para validar calidad y mantenibilidad

20. Relacione las métricas CK (como CBO o LCOM) con la mantenibilidad y calidad del software según ISO/IEC 25010.

Las métricas CK permiten medir atributos internos del software. Un CBO alto implica demasiado acoplamiento y reduce la modificabilidad; un LCOM alto indica baja cohesión y dificulta la mantenibilidad según ISO/IEC 25010

1. Explique cómo los microservicios ayudan a reducir la complejidad cognitiva en un sistema software.
2. ¿Por qué REST/JSON sustituyó en muchos casos a SOAP/WSDL en arquitecturas modernas?
3. Defina el concepto de “consistencia eventual” en microservicios y explique por qué aparece.
4. ¿Qué ventajas aporta la gobernanza descentralizada en arquitecturas de microservicios?
5. Explique cómo un servidor MCP puede mejorar la integración entre herramientas externas y modelos LLM.
6. ¿Qué significa “Smart Endpoints, Dumb Pipes” en una arquitectura de microservicios?
7. Defina “abstracción recurrente” y relacione el concepto con Ingeniería Directa e Ingeniería Inversa.
8. Explique cómo la Ingeniería Inversa ayuda a romper el “Vendor Lock-in”.
9. ¿Por qué el análisis de malware depende fuertemente de técnicas de ingeniería inversa?
10. Compare RISC-V y x86 desde el punto de vista de apertura tecnológica y análisis hardware.
11. ¿Qué papel desempeña OWASP Top 10 dentro de la seguridad de aplicaciones?
12. Explique la diferencia entre vulnerabilidades de tipo “Injection” y “Broken Access Control”.
13. ¿Por qué DevSecOps integra seguridad desde las primeras fases del desarrollo?

14. Relacione ISO/IEC 27034 con el ciclo S-SDLC.
15. Explique cómo Git y Markdown facilitan la filosofía Docs as Code.
16. ¿Qué problemas genera usar Word como sistema principal de documentación técnica?
17. Relacione RGPD con la responsabilidad del ingeniero software en el tratamiento de datos personales.
18. Explique la diferencia entre licencias permisivas y licencias copyleft.
19. ¿Por qué TDD se considera una estrategia de calidad preventiva y no reactiva?
20. Explique cómo las métricas de complejidad ciclomática ayudan a detectar deuda técnica y riesgos de mantenimiento.

1. **Microservicios y complejidad cognitiva**
Los microservicios reducen la complejidad cognitiva porque cada desarrollador solo necesita comprender la lógica interna de su servicio y la API de comunicación con otros sistemas, en lugar de entender toda la aplicación monolítica.
2. **REST/JSON vs SOAP/WSDL**
REST/JSON sustituyó en muchos casos a SOAP/WSDL porque es más ligero, flexible y eficiente. SOAP utiliza XML y contratos rígidos, generando mayor sobrecarga y complejidad.
3. **Consistencia eventual**
La consistencia eventual significa que los datos distribuidos entre microservicios pueden no sincronizarse instantáneamente, pero terminarán siendo consistentes con el tiempo. Surge porque cada servicio posee su propia base de datos.
4. **Gobernanza descentralizada**
Permite que cada equipo utilice el lenguaje, framework o base de datos más adecuada para su problema concreto, aumentando flexibilidad y autonomía tecnológica.
5. **Ventajas de MCP con LLMs**
MCP proporciona una comunicación estandarizada entre modelos LLM y herramientas externas, facilitando integración, interoperabilidad y automatización de tareas complejas.
6. **Smart Endpoints, Dumb Pipes**
Significa que la inteligencia y lógica de negocio reside dentro de cada microservicio (“Smart Endpoints”), mientras que la red solo transporta datos (“Dumb Pipes”).
7. **Abstracción recurrente**
La abstracción recurrente describe el flujo entre niveles de abstracción del software. La Ingeniería Directa transforma ideas abstractas en implementación, mientras que la Ingeniería Inversa parte de un sistema implementado para descubrir su diseño abstracto.
8. **Vendor Lock-in e Ingeniería Inversa**
La Ingeniería Inversa permite comprender sistemas propietarios para crear interoperabilidad o migrar tecnologías, reduciendo dependencia de un único proveedor.
9. **Ingeniería Inversa y malware**
El análisis de malware requiere desmontar y estudiar programas maliciosos para comprender su comportamiento, detectar vulnerabilidades y desarrollar defensas.
10. **RISC-V vs x86**
RISC-V es una arquitectura abierta y flexible, mientras que x86 es propietaria y más compleja. Esto hace que RISC-V facilite investigación, personalización y análisis hardware.
11. **Función de OWASP Top 10**
OWASP Top 10 identifica las vulnerabilidades web más críticas y sirve como referencia para priorizar medidas de seguridad en aplicaciones.
12. **Injection vs Broken Access Control**
Injection ocurre cuando un atacante introduce código malicioso en entradas del sistema (ej. SQL Injection). Broken Access Control ocurre cuando usuarios acceden a recursos o funciones sin permisos adecuados.
13. **DevSecOps y seguridad temprana**
DevSecOps incorpora seguridad desde las primeras fases del desarrollo para detectar vulnerabilidades antes de llegar a producción, reduciendo costes y riesgos.
14. **ISO 27034 y S-SDLC**
ISO 27034 proporciona el marco normativo de seguridad, mientras que S-SDLC aplica esos principios de forma práctica durante todo el ciclo de vida del software.

15. Git y Markdown en Docs as Code

Git permite control de versiones y colaboración, mientras Markdown facilita documentación ligera y portable. Juntos permiten tratar documentación igual que código fuente.

16. Problemas de Word en documentación técnica

Word dificulta control de versiones, colaboración simultánea, automatización CI/CD y trazabilidad de cambios, además de depender de formatos binarios propietarios.

17. RGPD y responsabilidad del ingeniero

El ingeniero software debe garantizar tratamiento seguro y legal de datos personales, aplicando principios como minimización de datos, privacidad y protección de información sensible.

18. Licencias permisivas vs copyleft

Las licencias permisivas permiten reutilizar código con pocas restricciones. Las licencias copyleft obligan a mantener abiertas las modificaciones derivadas del software original.

19. TDD como calidad preventiva

TDD es preventivo porque las pruebas se escriben antes del código, permitiendo detectar errores desde el inicio y evitando defectos tardíos.

20. Complejidad ciclomática y deuda técnica

La complejidad ciclomática mide la cantidad de caminos lógicos en el código. Valores altos indican software difícil de comprender, probar y mantener, aumentando deuda técnica y probabilidad de errores.

Resumen rápido de las ISO más importantes

ISO/IEC 25010 — Calidad del Software

Es el estándar más importante para evaluar la calidad de un producto software. Define las características que debe tener un software “bueno”.

¿Qué mide ISO 25010?

La norma divide la calidad en varias características:

- **Mantenibilidad** → facilidad para modificar el software.
- **Fiabilidad** → que funcione sin fallos.
- **Seguridad** → protección frente a ataques y accesos indebidos.
- **Usabilidad** → facilidad de uso.
- **Compatibilidad** → integración con otros sistemas.
- **Eficiencia** → buen rendimiento y consumo de recursos.
- **Portabilidad** → facilidad para moverlo a otros entornos.
- **Adecuación funcional** → que haga correctamente lo que se necesita.

Subcaracterísticas importantes

Dentro de mantenibilidad aparecen:

- Analizabilidad
- Testabilidad
- Modificabilidad
- Reusabilidad
- Modularidad

👉 Esta ISO sirve para medir si el software es mantenible, seguro y sostenible a largo plazo.

Otras ISOs importantes

ISO/IEC 27034 — Seguridad de Aplicaciones

Se centra en integrar seguridad durante todo el desarrollo software.

¿Qué hace?

- Define cómo desarrollar aplicaciones seguras.
- Introduce controles de seguridad en todas las fases.
- Reduce vulnerabilidades y ataques.

👉 Muy usada en DevSecOps y S-SDLC.

ISO/IEC 27005 — Gestión de Riesgos

Complementa la familia 27000.

¿Qué hace?

- Analiza riesgos de seguridad.
- Ayuda a identificar amenazas y vulnerabilidades.
- Define tratamientos y mitigaciones.

👉 Básica para ciberseguridad empresarial.

ISO/IEC 42001 — Gestión de IA

Primera ISO orientada a gobernanza de Inteligencia Artificial.

¿Qué hace?

- Regula el uso ético de IA.
- Controla riesgos y trazabilidad.
- Garantiza supervisión humana.

👉 Muy importante para IA generativa y LLMs.

ISO/IEC/IEEE 29119 — Testing Software

Norma internacional para pruebas de software.

¿Qué hace?

- Define procesos de testing.
- Estandariza documentación de pruebas.
- Organiza niveles de testing.

👉 Muy relacionada con TDD y QA.

IEEE 829 — Documentación de Testing

Estándar clásico de documentación de pruebas.

¿Qué hace?

- Define cómo documentar tests.
- Estructura planes e informes de pruebas.

👉 Predecesora de ISO 29119.

ISO/IEC 26514 — Documentación Software

Estándar para documentación técnica de usuario y sistemas.

¿Qué hace?

- Define documentación de calidad.
- Mejora mantenibilidad y comprensión.
- Ayuda a reducir dependencia del equipo.

👉 Muy ligada a “Docs as Code”.

ISO/IEC 5230 — Cumplimiento Open Source

Relacionada con licencias software.

¿Qué hace?

- Gestiona cumplimiento de licencias.
- Reduce riesgos legales.
- Controla uso de software open source.

👉 Muy importante en empresas grandes.

ISO 19506 (KDM) — Ingeniería Inversa

Estándar para representar conocimiento de sistemas legacy.

¿Qué hace?

- Analiza software antiguo.
- Facilita migraciones.
- Ayuda en ingeniería inversa.

👉 Muy útil para modernizar sistemas legacy.

La ISO de Calidad (25010) resumida MUY fácil

Característica	¿Qué controla?
Mantenibilidad	Facilidad de modificar
Fiabilidad	Que no falle
Seguridad	Protección
Usabilidad	Facilidad de uso
Compatibilidad	Integración
Eficiencia	Rendimiento
Portabilidad	Mover entre sistemas
Adecuación funcional	Que haga lo correcto

Lo más preguntado en examen

ISO 25010

➡ Calidad del software.

ISO 27034

➡ Seguridad en aplicaciones.

ISO 29119

➡ Testing.

ISO 42001

➡ Gobernanza IA.

ISO 26514

➡ Documentación software.

ISO 5230

➡ Licencias Open Source.

ISO 19506

➡ Ingeniería inversa / sistemas legacy.

1. **¿Código Auto-documentado o Estándar ISO?**

El código auto-documentado mejora la mantenibilidad, pero en sistemas críticos no es suficiente por sí solo. Normas como ISO/IEC 26514 exigen documentación formal para garantizar trazabilidad, auditoría y transferencia de conocimiento. En sectores críticos, la ley debe exigir manuales porque un buen código no sustituye responsabilidades legales y operativas.

2. **¿APIs para máquinas o interfaces para humanos?**

Aunque MCP y los LLMs puedan consumir APIs automáticamente, las interfaces humanas seguirán siendo necesarias para supervisión, validación y control. ISO/IEC 42001 enfatiza la supervisión humana en sistemas IA, evitando decisiones completamente autónomas. El futuro probablemente combinará APIs altamente estandarizadas con UIs más simples y estratégicas.

3. **¿Ghidra: filantropía o estrategia de la NSA?**

Ghidra puede verse como ambas cosas al mismo tiempo. La NSA liberó una herramienta extremadamente potente que beneficia a investigadores y profesionales de ciberseguridad, pero también fortalece el ecosistema defensivo estadounidense frente a amenazas globales. Estratégicamente, compartir herramientas puede aumentar la capacidad colectiva de análisis y seguridad nacional.

4. **¿Lanzar con vulnerabilidades “Medio” o bloquear despliegue?**

Depende del contexto de negocio y del riesgo operativo, pero ISO/IEC 27034 y OWASP recomiendan evaluar impacto antes de desplegar. Si las vulnerabilidades afectan datos sensibles o escalabilidad futura, debería bloquearse el despliegue. En entornos críticos, seguridad y reputación suelen pesar más que el time-to-market.

5. **¿Quién audita a la IA en Spec-Driven Development?**

La responsabilidad sigue siendo del ingeniero humano, aunque la IA genere el código automáticamente. ISO/IEC 25010 exige garantizar mantenibilidad, analizabilidad y testabilidad, algo que todavía requiere supervisión técnica humana. Más que desaparecer, el ingeniero evoluciona hacia un perfil híbrido: arquitecto, auditor y diseñador de especificaciones.

